

SKILL_WEEK-03

(SESSION NUMBER - 5)

Task 1: Apply Single Quotes to preserve literal content, ignore variable expansion, store quoted strings, validate parsing results, and test edge cases.

```
UW PICO 5.09

    {
        if (char_index > 0)
        {
            tokens[token_count][char_index] = '\0';
            token_count++;
            char_index = 0;
        }
    }
    else
    {
        tokens[token_count][char_index++] = input[i];
    }
}

if (in_single_quote)
{
    printf("Error: Unmatched single quote.\n");
    return 1;
}

if (char_index > 0)
{
    tokens[token_count][char_index] = '\0';
    token_count++;
}

printf("\nParsed Tokens:\n");

for (int i = 0; i < token_count; i++)
{
    printf("Token %d: [%s]\n", i + 1, tokens[i]);
}

return 0;
}
```

Fig 1: C program for parsing and preserving literal content within single quotes

Test case - 1:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill5_task1.c -o bin/skill5_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill5_task1
Enter command: echo 'Hello $USER'

Parsed Tokens:
Token 1: [echo]
Token 2: [Hello $USER]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 2: Parsing a single-quoted string while preserving literal content and ignoring variable expansion

Test case - 2:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill15_task1
Enter command: echo 'Hello World From OSSP'

Parsed Tokens:
Token 1: [echo]
Token 2: [Hello World From OSSP]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 3: Preserving spaces inside a single-quoted string during parsing

Test case - 3:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill15_task1
Enter command: echo 'Hello World
Error: Unmatched single quote.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 4: Detection of an unmatched single quote during command parsing



Fig 5: Flowchart for parsing and validating single-quoted strings

Result:

Single-quoted strings were parsed successfully while preserving spaces and

literal content. Variable expansion was ignored, quoted strings were stored as single tokens, and unmatched-quote edge cases were detected correctly.

Task 2: Apply Double Quotes to preserve spaces, allow variable expansion, parse nested tokens, validate outputs, and test quoted commands.

```
UW PICO 5.09

printf("Enter command: ");
fgets(input, sizeof(input), stdin);

input[strcspn(input, "\n")] = '\0';

for (int i = 0; input[i] != '\0'; i++)
{
    if (input[i] == '"')
    {
        in_double_quote = !in_double_quote;
    }
    else if (isspace((unsigned char)input[i]) && !in_double_quote)
    {
        if (char_index > 0)
        {
            tokens[token_count][char_index] = '\0';
            token_count++;
            char_index = 0;
        }
    }
    else
    {
        tokens[token_count][char_index++] = input[i];
    }
}

if (in_double_quote)
{
    printf("Error: Unmatched double quote.\n");
    return 1;
}

if (char_index > 0)
{
    tokens[token_count][char_index] = '\0';
    token_count++;
}

printf("\nParsed Tokens:\n");

for (int i = 0; i < token_count; i++)
{
    char expanded[MAX_LENGTH];

    expand_variable(tokens[i], expanded);

    printf("Token %d: [%s]\n", i + 1, expanded);
}

return 0;
}
```

Fig 6: C program for parsing double-quoted strings with variable expansion

Test Case - 1:

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill5_task2.c -o bin/skill5_task2
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill5_task2
Enter command: echo "Hello $USER"

Parsed Tokens:
Token 1: [echo]
Token 2: [Hello amrutha]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 7: Parsing a double-quoted string with variable expansion

Test case - 2:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill5_task2
Enter command: echo "Operating Systems Skill Five"

Parsed Tokens:
Token 1: [echo]
Token 2: [Operating Systems Skill Five]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 8: Preserving spaces inside a double-quoted string

Test case - 3:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill5_task2
Enter command: echo "Hello World
Error: Unmatched double quote.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 9: Detection of an unmatched double quote during parsing

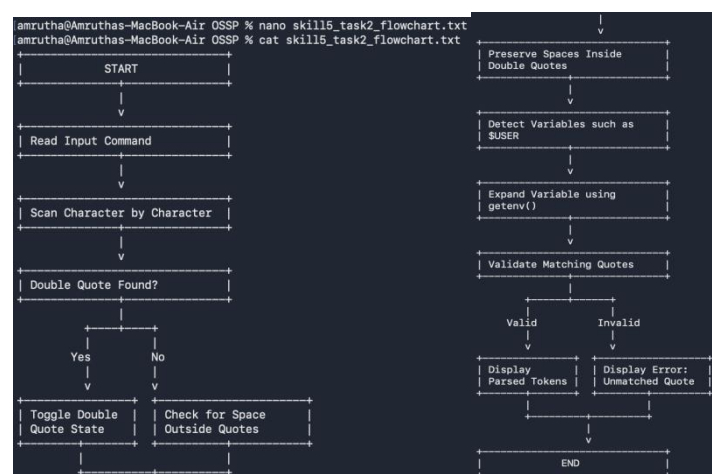


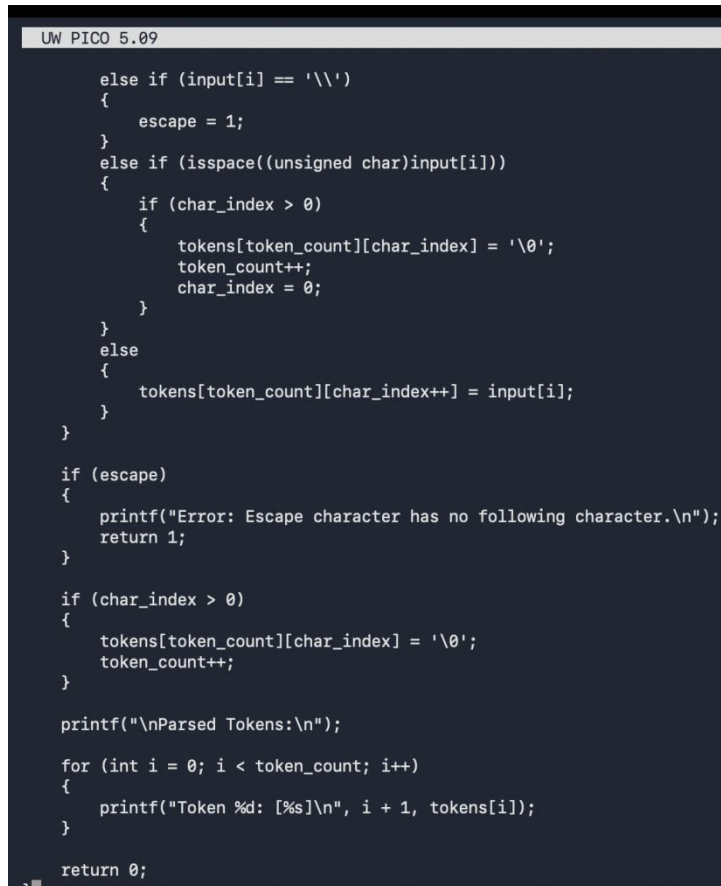
Fig 10: Flowchart for parsing double-quoted strings with variable expansion

Result:

Double-quoted strings were parsed successfully while preserving spaces and allowing variable expansion. Environment variables such as \$USER were expanded correctly, and unmatched double-quote cases were detected successfully.

(SESSION NUMBER - 6)

Task 1: Create process escape sequences, handle escaped spaces, escape special symbols, preserve characters, validate parser output, and test complex inputs.

A screenshot of a code editor showing a C program. The program is titled "UW PICO 5.09". It contains a function that processes input characters. It handles escaped backslashes, spaces, and other special characters. It uses a 'tokens' array to store parsed tokens and a 'token_count' variable to track the number of tokens. The program prints the parsed tokens and returns 0.

```
UW PICO 5.09

    else if (input[i] == '\\')
    {
        escape = 1;
    }
    else if (isspace((unsigned char)input[i]))
    {
        if (char_index > 0)
        {
            tokens[token_count][char_index] = '\\0';
            token_count++;
            char_index = 0;
        }
    }
    else
    {
        tokens[token_count][char_index++] = input[i];
    }
}

if (escape)
{
    printf("Error: Escape character has no following character.\n");
    return 1;
}

if (char_index > 0)
{
    tokens[token_count][char_index] = '\\0';
    token_count++;
}

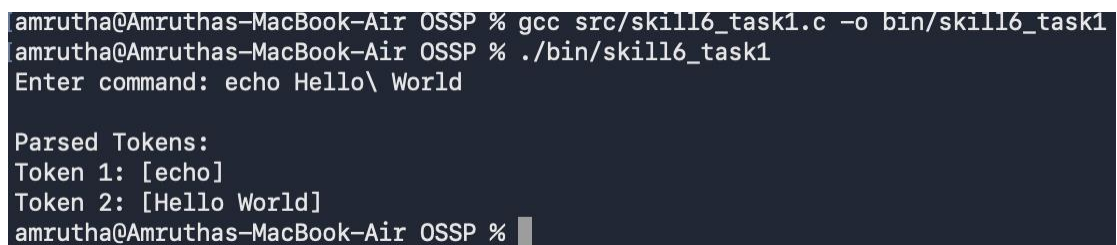
printf("\nParsed Tokens:\n");

for (int i = 0; i < token_count; i++)
{
    printf("Token %d: [%s]\n", i + 1, tokens[i]);
}

return 0;
```

Fig 1: C program for parsing escaped spaces and special characters in command input

Test case - 1:

A screenshot of a terminal window showing the execution of the program. The user enters the command "echo Hello\ World". The program outputs "Parsed Tokens:" followed by "Token 1: [echo]" and "Token 2: [Hello World]".

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill6_task1.c -o bin/skill6_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill6_task1
Enter command: echo Hello\ World

Parsed Tokens:
Token 1: [echo]
Token 2: [Hello World]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 2: Parsing an escaped space while preserving the complete string as one token

Test Case - 2:

```

amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill6_task1
Enter command: echo \$USER

Parsed Tokens:
Token 1: [echo]
Token 2: [$USER]
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 3: Preserving an escaped special symbol during command parsing

Test Case - 3:

```

[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill6_task1
Enter command: echo hello\&world

Parsed Tokens:
Token 1: [echo]
Token 2: [hello&world]
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 4: Parsing escaped special characters as literal content

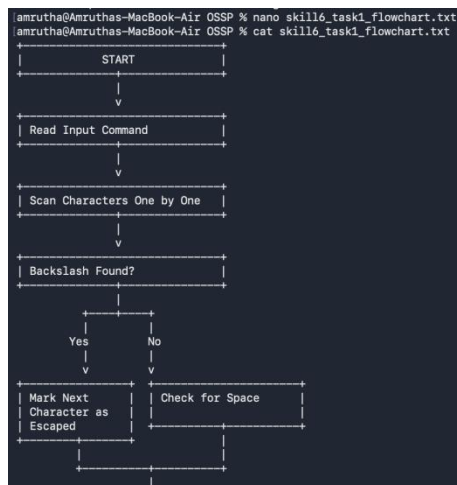
Test Case - 4:

```

[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill6_task1
Enter command: echo Hello\
Error: Escape character has no following character.
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 5: Detection of an incomplete escape sequence at the end of the command



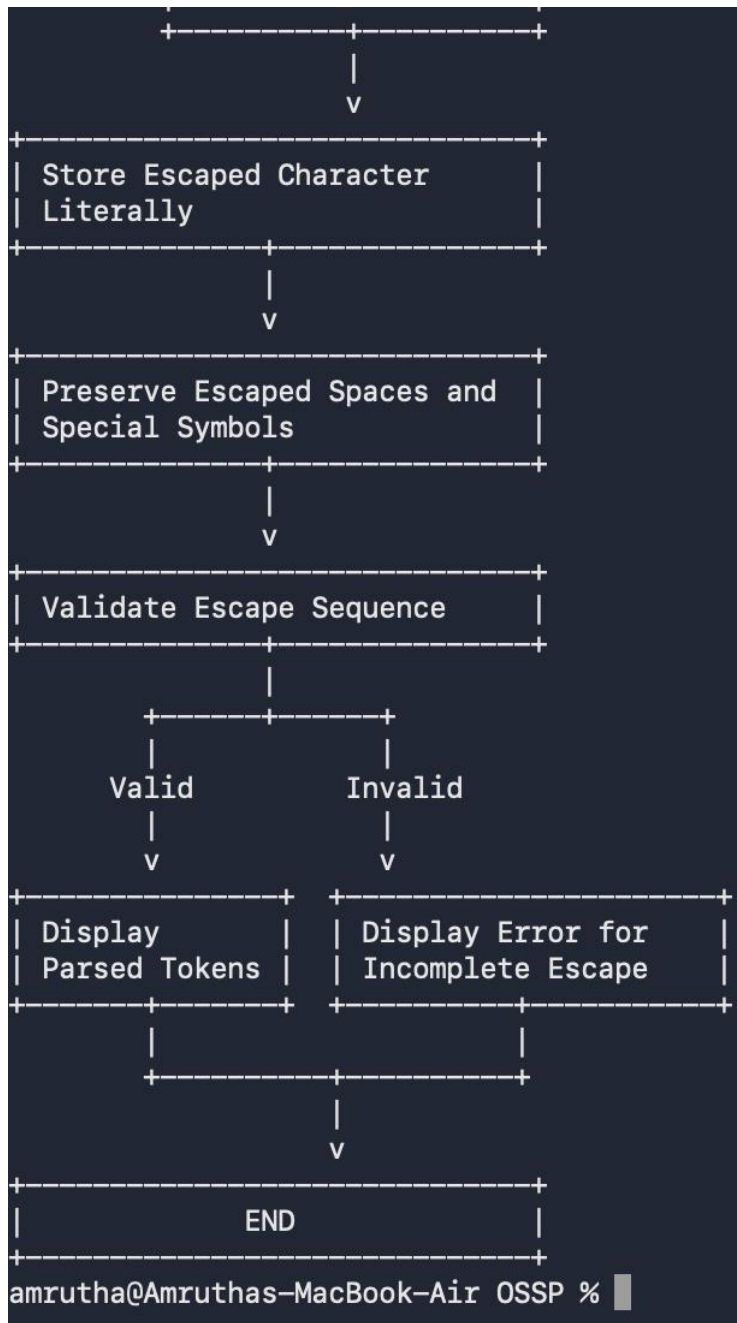


Fig 6: Flowchart for parsing escaped spaces and special characters

Result:

Escape sequences were processed successfully. Escaped spaces and special symbols were preserved as literal characters, complex inputs were parsed correctly, and incomplete escape sequences were detected successfully.

Task 2: Create child processes, execute programs, handle execution errors, pass arguments, manage the parent process, and test command launching.

```
UW PICO 5.09

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent Process PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());
        printf("Executing command: ls -l\n");

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }
    else
    {
        printf("Parent waiting for child process...\n");

        wait(NULL);

        printf("Child process completed.\n");
        printf("Parent process terminating.\n");
    }

    return 0;
}
```

Fig 7: C program for creating a child process and executing a command using fork() and exec()

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill6_task2.c -o bin/skill6_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill6_task2
Parent Process PID: 63076
Parent waiting for child process...
Child Process PID: 63081
Executing command: ls -l
total 168
drwxr-xr-x  3 amrutha  staff   96 Jul 31 14:05 assets
drwxr-xr-x 27 amrutha  staff  864 Sep 21 11:37 bin
drwxr-xr-x  4 amrutha  staff  128 Jul 31 14:05 docs
-rw-r--r--  1 amrutha  staff 1507 Aug 14 19:15 double_quote_flowchart.txt
-rw-r--r--  1 amrutha  staff 1967 Aug 15 10:40 escape_flowchart.txt
-rw-r--r--  1 amrutha  staff   30 Aug 13 19:58 file1.txt
-rw-r--r--  1 amrutha  staff   30 Aug 13 20:04 file2.txt
drwxr-xr-x  6 amrutha  staff  192 Jul 31 14:05 include
-rw-r--r--  1 amrutha  staff   10 Aug 13 18:15 LICENSE
-rw-r--r--  1 amrutha  staff   10 Aug 13 19:15 Makefile
drwxr-xr-x  2 amrutha  staff   64 Jul 31 13:59 obj
-rw-r--r--  1 amrutha  staff 1177 Aug 14 17:11 parser_flowchart.txt
-rw-r--r--  1 amrutha  staff 1207 Aug 15 15:01 process_flowchart.txt
-rw-r--r--  1 amrutha  staff 1277 Aug 14 17:42 quote_flowchart.txt
-rw-r--r--  1 amrutha  staff   10 Aug 13 18:15 README.md
drwxr-xr-x  3 amrutha  staff   96 Jul 31 14:05 scripts
-rw-r--r--  1 amrutha  staff 1880 Sep 21 11:00 skill5_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff 2013 Sep 21 11:12 skill5_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff 1902 Sep 21 11:32 skill6_task1_flowchart.txt
drwxr-xr-x 32 amrutha  staff 1024 Sep 21 11:37 src
drwxr-xr-x  4 amrutha  staff  128 Jul 31 14:05 tests
-rw-r--r--  1 amrutha  staff 1702 Aug 17 10:43 wait_flowchart.txt
-rw-r--r--  1 amrutha  staff 1848 Aug 24 20:52 week3_task3_flowchart.txt
-rw-r--r--  1 amrutha  staff 1246 Aug 24 21:20 week3_task4_flowchart.txt
-rw-r--r--  1 amrutha  staff 2036 Sep 19 14:06 week5_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff 1760 Sep 19 14:17 week5_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff 1760 Sep 19 14:52 week6_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff 1520 Sep 19 15:14 week6_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff 1594 Sep 19 17:26 week7_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff 1494 Sep 19 17:53 week7_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff 1328 Sep 20 21:38 week8_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff 1726 Sep 20 21:48 week8_task2_flowchart.txt
Child process completed.
Parent process terminating.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 8: Successful child process creation and execution of the ls -l command using fork() and exec()

```

amrutha@Amruthas-MacBook-Air OSSP % nano src/skill6_task2.c
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill6_task2.c -o bin/skill6_task2
./bin/skill6_task2
Parent Process PID: 63822
Parent waiting for child process...
Child Process PID: 63823
Executing command: ls -l
exec failed: No such file or directory
Child process completed.
Parent process terminating.
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 9: Handling command execution failure when `exec()` is given an invalid program name.

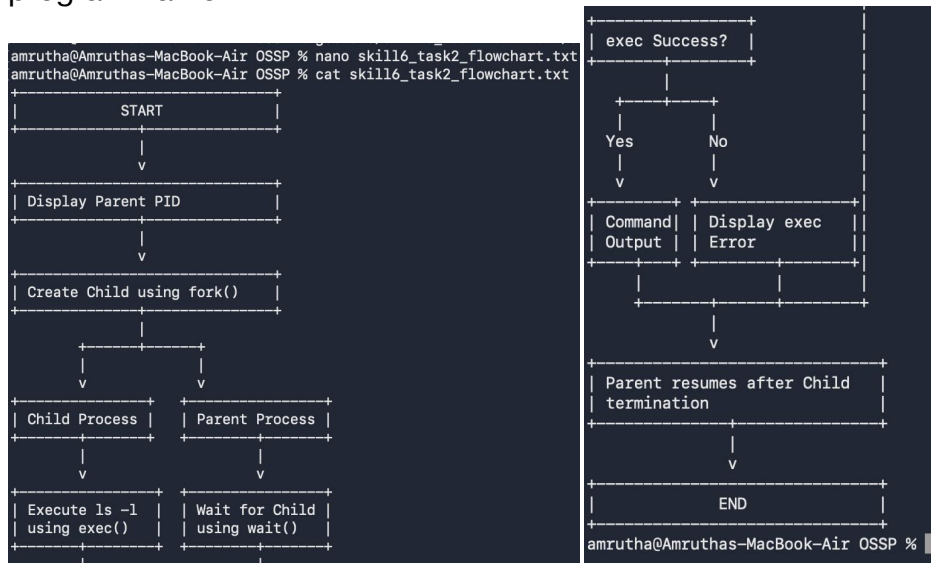


Fig 10: Flowchart for child process creation and command execution using `fork()`, `exec()`, and `wait()`

Result:

A child process was created successfully using `fork()`, and the `ls -l` command was executed using `exec()`. Command-line arguments were passed correctly, the parent process waited for the child using `wait()`, and execution errors were handled successfully for invalid commands.